

מרים — פאה חדשה (New Wigs)

הזמנת פאה חדשה, זרימת ייצור ותחנת עבודה

מפתחת 2 · New Wigs Module

1. פתיחה — מה לומר (30 שניות)

"שלום, אני מרים. אחראית על מודול הפאה החדשה — מהזמנת פאה חדשה ועד סיום הייצור. בנתי את טופס ההזמנה, את תחנת הייצור לעובדות, ואת זרימת 6 השלבים שמעבירה כל פאה מהתאמת שיער ועד בקרת איכות."

2. מה עשיתי

- טופס הזמנת פאה חדשה — 3 שלבים: זיהוי לקוחה, רישום מהיר, מפרט טכני מלא.
- שיבוץ עובדות לכל שלב — עם תאריכי יעד לכל שלב ייצור.
- זרימת 6 שלבים — התאמת שיער → תפירה → צבע → עבודת יד → חפיפה → בקרה.
- תחנת ייצור (Production Station) — עובדות רואות משימות ומסמנות "בוצע".
- כרטיס מפרט טכני (Technical Card) — עובדות רואות מפרט מלא (ללא פרטי תשלום).
- חתימה דיגיטלית + צילום + PDF — בסיום ההזמנה נשלח מייל עם סיכום עסקה.
- אינטגרציה עם QA — בסיום ייצור נוצרת משימת בקרת איכות אוטומטית.

3. קבצים מרכזיים

שכבה	קובץ	תפקיד
Client	components/NewWigs/NewOrderForm/NewOrderForm.tsx	טופס הזמנה (797 שורות)
Client	components/NewWigs/ProductionStation/ProductionStation.tsx	תחנת ייצור לעובדות
Client	components/NewWigs/WigTechnicalCard/WigTechnicalCard.tsx	כרטיס מפרט טכני
Server	Models_Service/NewWigs/newWigModel.ts	מודל — MongoDB NewWig
Server	Models_Service/NewWigs/newWigService.ts	לוגיקת שלבים + QA
Server	Routers/newWigRouter.ts	API — /api/wigs
Tests	tests/newWig.test.ts	בדיקות CRUD, שלבים, דחיפות
Tests	tests/e2e_workflow.test.ts	זרימה מלאה עד QA ופסילה

4. הסבר על קטעי קוד

קטע 1 — שליחת הזמנה חדשה (Client)

קובץ: NewOrderForm.tsx — פונקציית onSubmit

```
const onSubmit = async (data) => {
  if (!signatureData) { alert('חובה להחתים את הלקוחה!'); return; }
  const firstStageWorkers = plannedAssignments['התאמת שיער'] || [];
  if (firstStageWorkers.length === 0) {
    alert("חובה לשבץ לפחות עובדת אחת לשלב 'התאמת שיער'");
    return;
  }

  const payload = {
    ...data,
    orderCode: data.orderCode.trim(),
    customer: finalCustomerId,
    assignedWorkers: firstStageWorkers,
    stageAssignments: plannedAssignments,
    stageDeadlines: stageDeadlines,
    currentStage: 'התאמת שיער',
    customerSignature: signatureData,
    imageUrl: capturedImage,
    internalNote: internalNote
  };

  const response = await axios.post('wigs/new', payload);
};
```

מה קורה כאן:

- לפני שליחה — בודקים שיש **חתימת לקוחה** ושיבוץ לשלב הראשון.
- ה-Payload כולל: מפרט טכני, שיבוץ עובדות לכל שלב, תאריכי יעד, חתימה ותמונה.
- השלב הראשון תמיד: **"התאמת שיער"**.
- הנתונים נשלחים ל-POST /api/wigs/new ונשמרים ב-MongoDB.

קטע 2 — זרימת השלבים (Server)

קובץ: STAGES_FLOW — newWigService.ts

```
const STAGES_FLOW = [
  'התאמת שיער',
  'תפירת פאה',
  'צבע',
  'עבודת יד',
  'חפיפה',
  'בקרה'
];
```

```
// כשעובדת לוחצת "בוצע":
const currentIndex = STAGES_FLOW.findIndex(s => s.trim() === wig.currentStage.trim());
if (currentIndex === STAGES_FLOW.length - 2) {
  nextStage = 'בקרה';
  isMovingToQA = true;
} else {
  nextStage = STAGES_FLOW[currentIndex + 1];
}
```

מה קורה כאן:

- כל פאה עוברת 6 שלבים בסדר קבוע.
- כשעובדת לוחצת "בוצע", `moveToNextStage()` מוצאת את השלב הבא במערך.
- רושמת בהיסטוריה מי סיימה ומתי, ומשבצת את העובדת הבאה.
- בשלב האחרון לפני בקרה — יוצרת **משימת QA** אוטומטית.

קטע 3 — יצירת QA אוטומטית (Server)

קובץ: `newWigService.ts` — בסיום ייצור

```
if (isMovingToQA || nextStage === 'בקרה') {
  await Service.create({
    customer: wig.customer,
    serviceType: 'Production QA',
    origin: 'NewWig',
    newWigReference: wig._id,
    status: 'QA',
    notes: { secretary: 'ייצור: איכות מסיום ייצור' }
  });

  return await NewWig.findByIdAndUpdate(wigId, {
    currentStage: 'בקרה',
    assignedWorkers: [],
    $push: { history: historyEntry }
  });
}
```

מה קורה כאן:

- כשהפאה מסיימת את כל שלבי הייצור — נוצרת רשומת Service עם `'status: QA'`.
- חני (מודול QA) רואה את המשימה בלוח הבקרה שלה (`qa/`).
- הפאה עוברת לסטטוס "**בקרה**" וממתינה לאישור.

5. תסריט הדגמה

1 התחברי כ- admin / password123 → מגיעים ל- / (טופס הזמנה).

2 חפשי לקוחה לפי ת.ז. (למשל: 329520449 — מרים גליק).

3 מלאי מפרט טכני: מידות, סוג שיער, צבע, סגנון קדמי.

4 שבצי עובדות לכל שלב ייצור (שרה, ליפשי, הודיה...).

5 חתמי את הלקוחה דיגיטלית + צלמי אותה.

6 שלחי — הודעת הצלחה + PDF נשלח למייל הסלון.

7 עברי ל- production/ — הציגי את המשימה אצל העובדת המשובצת.

8 לחצי "בוצע" — הסבירי שהפאה עוברת לשלב הבא.

6. טכנולוגיות

Express Router

Mongoose

Axios

html2canvas + jsPDF

TypeScript

React Hook Form

Jest

7. שאלות ותשובות למורה

ש: מה 6 שלבי הייצור?

1. התאמת שיער — 2. תפירת פאה — 3. צבע — 4. עבודת יד — 5. חפיפה — 6. בקרה. השלבים מוגדרים במערך STAGES_FLOW ב-newWigService.ts. כל שלב משובץ לעובדת עם התמחות מתאימה.

ש: למה חובה חתימת לקוחה?

החתימה הדיגיטלית (customerSignature) נשמרת כתמונת Base64 ב-DB. היא מהווה אישור הלקוחה על המפרט הטכני והמחיר. בלי חתימה — הטופס לא נשלח (client validation). זה גם נכלל ב-PDF שנשלח למייל הסלון.

ש: מה קורה כש-QA פוסל פאה?

חני פוסלת דרך RejectionModal — בוחרת שלבים להחזרה (למשל: צבע, עבודת יד). השרת שומר את השלבים ב-pendingRepairStages בפאה. moveToNextStage מזהה שיש שלבים ממתינים ומחזיר את הפאה לשלב הראשון ברשימה, עם qaNote שמוצג לעובדת בכרטיס המפרט הטכני.

ש: איך עובדת שיבוץ העובדות?

בזמן ההזמנה, המזכירה בוחרת עובדות לכל שלב ושומרת ב-stageAssignments (Map). כשפאה עוברת לשלב הבא, השרת מחפש את העובדת המשובצת מראש. אם לא שובצה — הוא מחפש עובדת לפי specialty (התמחות) ב-DB.

ש: מה ההבדל בין NewOrderForm ל-ProductionStation?

NewOrderForm — מיועד למזכירה: יצירת הזמנה חדשה, מפרט מלא, תשלום, חתימה.
ProductionStation — מיועד לעובדות: רואות רק את המשימות שלהן, מסמנות "בוצע", רואות כרטיס מפרט טכני (בלי פרטי תשלום). שני המסכים עובדים על אותו מודל NewWig.

ש: איך נוצר ה-PDF?

אחרי שליחת ההזמנה, הקוד משתמש ב-`html2canvas` לצילום הטופס כתמונה, ואז ב-`jsPDF` ליצירת קובץ PDF. ה-PDF נשלח למייל הסלון דרך `POST /api/wigs/send-summary-email`.

ש: מה קורה בלחיצה כפולה על "בוצע"?

ב-`moveToNextStage` יש בדיקה: אם הפאה כבר ב-בקרה או נמסר, הפונקציה מחזירה את הפאה בלי שגיאה (idempotent). זה מונע באגים מלחיצה כפולה או מפקודה קולית כפולה.

ש: אילו בדיקות כתבת?

`newWig.test.ts` — יצירת פאה, שליפת משימות לעובדת, שינוי דחיפות, מעבר שלב.
`e2e_workflow.test.ts` — זרימה מלאה: יצירה → כל השלבים → QA → פסילה → חזרה לייצור.